

BENDER: A World-Model Coprocessor Architecture for Large Language Models

Author: Rich Schiavi Date: 2026

Abstract

Large Language Models (LLMs) exhibit strong natural language capabilities but rely on implicit world models internalized during training — representations that are opaque, static, and difficult to audit. Prompt-side retrieval methods such as RAG and GraphRAG partially address knowledge staleness, but they remain constrained by context window costs, the "lost in the middle" degradation effect, and the model's continued need to reconstruct structure from unstructured text.

We propose **BENDER** (Bidirectional Entity Network and Deterministic Execution Runtime), a World-Model Coprocessor architecture that separates *structured runtime reasoning* from *probabilistic language generation*. BENDER maintains an explicit, typed knowledge graph and a parallel cognitive runtime that performs retrieval, planning, rule application, and simulation. Its primary output is a **Control Packet** — a structured object containing entities, constraints, hypotheses, and a **fused latent vector** designed for direct injection into the model's inference path.

We demonstrate the viability of BENDER's structured control mode through a first-place result on the Spider 2 text-to-SQL benchmark, achieved via integration with the ScalarLM/vLLM serving stack. We further argue that BENDER's explicit provenance and deterministic reasoning pipeline offer natural affordances for AI safety properties — including auditability, interpretability, and hard constraint enforcement — that probabilistic generation alone cannot provide. Native latent integration (Level 2/3) is presented as the primary research objective, with the current implementation constituting a working Level 1 baseline.

1. Introduction

The dominant paradigm for LLM deployment treats the model's internalized world knowledge as both the reasoning substrate and the knowledge store. This conflation imposes hard limits: knowledge frozen at training time cannot be corrected without retraining; reasoning steps are encoded in activations that resist interpretation; and probabilistic generation cannot enforce hard logical or domain constraints.

Retrieval-Augmented Generation (RAG) and its graph-structured variant (GraphRAG) address the staleness problem by injecting retrieved text into the prompt. But they leave the structural

problem intact. The model must still reconstruct typed relations, constraint hierarchies, and provenance chains from unstructured token sequences — a task it was not explicitly trained to perform reliably.

BENDER takes a different position: that a **coprocessor architecture**, running a deterministic typed runtime in parallel with inference, can supply structured world-model state directly to the model's latent stream. This eliminates the serialization overhead of prompt-side retrieval, enables hard constraint enforcement, and produces reasoning traces that are auditable by construction.

This paper makes the following contributions:

1. A formal description of the BENDER coprocessor architecture and its cognitive lifecycle.
 2. A characterization of three integration levels (prompt-side, orchestration-side, and native latent) and their tradeoffs.
 3. A positioning of BENDER against related architectures — including prefix tuning, Flamingo, RETRO, and ToolFormer — that clarifies what is novel about the coprocessor approach.
 4. An argument that BENDER's design properties are aligned with AI safety goals, particularly around auditability, consistency, and scalable oversight.
 5. A description of the ScalarLM/vLLM integration strategy, grounded in a demonstrated benchmark result, and the research path toward native latent integration.
-

2. The Problem: Implicit World Models Are Weak Interfaces

Implicit world models in LLMs exhibit four primary failure modes:

Staleness. Training-time knowledge is fixed. Rapidly evolving domains — clinical evidence, regulatory databases, instrument-specific calibrations — cannot be updated without retraining or fine-tuning.

Opacity. Reasoning is encoded in high-dimensional activations with no explicit trace. This makes provenance attribution and error auditing fundamentally difficult, a recognized challenge for deployed AI systems.

Inconsistency. Probabilistic generation cannot enforce hard constraints. A model may simultaneously assert contradictory facts across a multi-step reasoning chain, with no internal mechanism to detect or correct the contradiction.

Context Bottleneck. Injecting large-scale domain knowledge as prompt text is expensive in tokens and subject to well-documented positional degradation effects. Relevant information placed in the middle of long contexts is systematically underweighted during attention.

These limitations are not incidental. They are structural consequences of treating a language model as both the reasoning engine and the knowledge store.

3. Related Work

BENDER's design intersects with several existing lines of research. We describe each and clarify the distinctions.

Retrieval-Augmented Generation (RAG). RAG systems retrieve relevant documents and inject them as prompt text prior to generation. GraphRAG extends this to graph-structured retrieval. Both are *pre-inference* text transformations. BENDER differs in that it operates as an *intra-inference* intervention, injecting structured state — not text — into the model's forward pass. Additionally, RAG does not maintain persistent typed world state; each call retrieves independently.

Prefix Tuning and Soft Prompts. Li and Liang (2021) and subsequent work demonstrated that prepending learned continuous vectors to the input can steer model behavior without full fine-tuning. BENDER's fused vector injection is architecturally related, but differs in a critical respect: prefix tuning vectors are *trained parameters*, fixed after optimization. BENDER's fused vector is *dynamically constructed at inference time* from a runtime world model, making it request-scoped and updateable without any gradient computation.

Flamingo. Alayrac et al. (2022) introduced cross-attention layers interleaved with frozen LLM layers, enabling vision-language grounding. BENDER's cross-attention injection pathway is inspired by this design. The distinction is domain and modality: Flamingo conditions on visual features from a frozen vision encoder; BENDER conditions on symbolic world-model state — typed entities, constraint sets, and simulation outputs — that can be updated dynamically and maintained across inference requests.

RETRO. Borgeaud et al. (2022) proposed chunked cross-attention over a large retrieval database, enabling kNN-augmented generation at scale. RETRO retrieves text chunks and attends over them via dedicated cross-attention layers. BENDER's approach differs in two ways: it operates over typed graph structures rather than text chunks, and its runtime applies deterministic reasoning (rule application, simulation, planning) before the fusion step, rather than passing retrieved text directly to the model.

ToolFormer and Function Calling. Schick et al. (2023) showed that LLMs can learn to call external APIs by inserting call tokens into generation. Modern function-calling frameworks extend this with structured schemas. These approaches are *generation-side*: the model decides when and how to call tools by generating tokens. BENDER operates at a lower level — the coprocessor influences the model's *latent state* before token generation decisions are made, enabling constraint enforcement that does not depend on the model voluntarily invoking a tool.

Knowledge Graph Embedding Methods. TransE, RotatE, and related methods embed graph structure into vector spaces for downstream tasks. BENDER uses typed knowledge graphs as its world model but does not rely on pre-trained graph embeddings as its primary interface. The FusionModule constructs the fused vector dynamically from active runtime state, which may include embedding-based similarity but is not reducible to it.

Summary. BENDER is most closely related to Flamingo (cross-attention injection) and RETRO (large-scale retrieval into model layers), but is distinguished by: (1) its use of typed symbolic world-model state rather than text or image features, (2) its deterministic reasoning pipeline applied before fusion, (3) its request-scoped dynamic vector construction, and (4) its explicit provenance and auditability properties.

4. The BENDER Architecture

BENDER is structured as a coprocessor loop running in parallel with LLM inference.

4.1 Components

World Model. A persistent, typed knowledge graph with nodes (entities) and directed typed edges (relations). Nodes carry attribute schemas; edges carry provenance metadata. The world model is an external asset, separate from the coprocessor runtime, and can in principle be reused across model backends.

BenderRuntime. The orchestration engine managing the cognitive lifecycle of a request. Stateless across requests; all state is passed explicitly via the Control Packet.

ModelIntegration. An interface for capturing model-side state (query embedding, optional hidden states) and injecting coprocessor-side state (fused vector, constraint signals).

Planner. Generates a deterministic reasoning plan from the query and retrieved world-model state. Plans are explicit, inspectable sequences of operations.

RuleEngine. Applies domain-specific hard constraints and soft preferences to the active state. Constraints are encoded as typed rules, not implicit model behaviors.

Simulator. Performs lightweight forward simulation to test hypotheses or predict outcomes before generation. Enables the coprocessor to reject implausible hypotheses without consuming model compute.

FusionModule. Combines model-side embeddings and graph-side structured state into a fused latent vector for injection into the model's forward pass.

4.2 The Cognitive Lifecycle

A BENDER request proceeds through seven stages:

1. **Capture.** The query embedding (and optionally intermediate hidden states) is captured from the model's inference path.
2. **Retrieve.** Relevant nodes and edges are fetched from the World Model based on the query embedding and structured filters.
3. **Plan.** The Planner constructs a deterministic reasoning plan addressing the query given retrieved state.
4. **Apply Rules.** The RuleEngine enforces hard constraints and applies domain logic to the active entity set.
5. **Simulate.** The Simulator tests plausible hypotheses, pruning candidates that fail hard constraints before generation.
6. **Fuse.** The FusionModule constructs a fused latent vector combining model-side and graph-side signals.
7. **Inject.** A Control Packet is returned to the inference server for integration into the model's forward pass.

4.3 The Control Packet

The Control Packet is the coprocessor's primary output. It contains:

- **Active entity set:** typed nodes retrieved and validated during the cognitive lifecycle
 - **Active relation set:** edges relevant to the current query
 - **Constraint set:** hard constraints derived by the RuleEngine
 - **Hypothesis set:** candidate explanations generated and scored by the Simulator
 - **Provenance chain:** explicit trace of the reasoning steps that produced the packet
 - **Fused vector:** a dense numerical representation of the coprocessor's structured opinion on the current state, intended for injection into the model's latent stream
-

5. Integration Levels

BENDER defines three integration levels with distinct capability and implementation requirements.

Level 1: Structured Control Mode. The Control Packet is serialized and injected as structured prompt context or passed to an orchestration layer (e.g., a LangChain or DSPy pipeline). This is the current implemented baseline. It provides structured retrieval, rule enforcement, and provenance at the cost of prompt tokens and without native latent influence. This is the mode used in the Spider 2 benchmark work described in Section 7.

Level 2: Orchestration-Side Native. The fused vector is injected into the model's inference path via an adapter framework (e.g., LoRA or Tokenformer) that is activated per-request without modifying the base model weights. This requires cooperation from the inference server but does not require architectural changes to the base model.

Level 3: Request-Scoped Native Integration. The fused vector is injected directly into the model's residual stream or via dedicated cross-attention layers during the forward pass, on a per-request basis. This requires backend support such as the ScalarLM/vLLM fork described in Section 8. It is the target state for full BENDER deployment and the subject of ongoing research.

It is important to distinguish these levels operationally. Benchmark results derived from Level 1 should not be described as demonstrating native coprocessor integration. The architectural claims about efficiency and expressiveness of the fused vector apply specifically to Level 2 and Level 3.

6. BENDER and AI Safety

A coprocessor architecture with explicit typed world state has natural affordances for AI safety properties that are difficult to achieve with probabilistic generation alone.

Auditability. The provenance chain in the Control Packet provides an explicit, inspectable trace of the reasoning steps that influenced a model output. This is directly relevant to mechanistic interpretability: rather than post-hoc attribution of model behavior to internal activations, BENDER records the *intended* structured influence as a first-class artifact.

Hard Constraint Enforcement. The RuleEngine applies domain-specific constraints before generation. This enables a class of safety properties — "the model cannot assert X given constraint Y" — that prompt-side instructions cannot reliably enforce. A constraint encoded in the RuleEngine is enforced deterministically; a constraint expressed in a system prompt is a probabilistic suggestion.

Consistency. By maintaining persistent typed world state across reasoning steps, BENDER can detect and flag contradictions before they propagate into generated text. This addresses a known failure mode of long-context chain-of-thought reasoning in current LLMs.

Scalable Oversight. As AI systems operate in higher-stakes domains, human oversight depends on the ability to understand and verify system reasoning. BENDER's explicit plan and provenance structures are designed to support human-in-the-loop verification at a granularity that token-level generation cannot provide.

These properties are not incidental to the architecture. They are direct consequences of the coprocessor separation: when reasoning is performed by a deterministic typed runtime, it is auditable, constrainable, and correctable in ways that are structurally unavailable to implicit neural reasoning.

7. Domain Instantiations and Stress Tests

BENDER's runtime is domain-agnostic. Domain knowledge is encapsulated in World Model implementations that sit above the core runtime. We describe three instantiations of increasing complexity.

7.1 Spider 2: Schema Grounding and Execution-Time Repair

Spider 2.0 is a challenging text-to-SQL benchmark requiring multi-step schema grounding, query planning, and execution-time repair of failed queries. It serves as a concrete stress test for BENDER's Planner and RuleEngine components.

Using BENDER's structured control mode (Level 1) integrated with ScalarLM/vLLM, we achieved first place on the Spider 2 benchmark. The BENDER runtime contributed schema-grounded entity resolution and execution-time repair logic that extended the base LLM's SQL generation capabilities. This result establishes the viability of the Level 1 baseline and motivates the research path toward native integration.

7.2 Biomedical Interpretation

A biomedical world model demonstrates BENDER's value in a knowledge-dense, rapidly evolving domain. Requirements include:

- Normalization of biomarkers, diseases, assays, targets, and therapies to canonical identifiers
- Provenance preservation across heterogeneous evidence sources
- Hypothesis derivation from explicit entity-relation structure rather than retrieved text

This domain is strategically significant because the evidence state changes continuously (new clinical trials, regulatory decisions, literature) and is too specialized to justify full model retraining for every update. The BENDER world model can be updated incrementally without touching the base model.

7.3 Mass Spectrometry Interpretation

Mass spectrometry is a particularly strong test case because the reasoning it requires is fundamentally incompatible with approximate probabilistic generation:

- **Hard mass constraints:** candidate molecular formulas must satisfy exact mass arithmetic within instrument tolerance
- **Polarity and adduct compatibility:** ionization constraints eliminate candidates regardless of semantic plausibility

- **Instrument- and sample-context-aware reasoning:** interpretation depends on experimental metadata that is not encodable in text prompts efficiently
- **Provenance on candidates:** each candidate explanation requires an attached evidence chain for downstream validation

This domain constitutes a principled counterexample to the claim that vector database retrieval plus RAG is sufficient for specialized scientific interpretation. The constraints are not soft preferences; they are hard physical laws. BENDER's RuleEngine is the appropriate place to enforce them.

8. Integration with ScalarLM and vLLM

The target integration path for Level 2 and Level 3 BENDER is ScalarLM's `vllm-fork`, which provides the necessary hooks for request-scoped inference modification.

8.1 Architectural Components

Tokenformer Surgeon. Wraps late MLP or attention projection layers to accept coprocessor residuals. The fused vector is added to the layer output without modifying base model weights, enabling per-request injection without global weight swaps.

ActiveBenderBatch. Manages per-request coprocessor contexts within a scheduled inference batch. Each request in a batch may carry a distinct Control Packet; the ActiveBenderBatch ensures that the correct fused vector is applied to the correct request during the forward pass.

Worker-Side Activation. Injects the fused vector into the forward pass at the worker level, below the scheduler abstraction. This enables coprocessor influence without requiring changes to the vLLM scheduling or tokenization pipeline.

8.2 Integration with the Dijkstra Architecture

BENDER's Level 3 integration path is architecturally complementary to the Dijkstra cognitive ensemble system. In Dijkstra, the Tokenformer replaces dense weight projection with dynamic SSD-resident weight retrieval via RDMA. BENDER extends this by determining *what* is retrieved and *how* it is fused:

- The BENDER World Model resides on the SSD substrate alongside Dijkstra's parameter library
- The Cluster Traversal Unit (CTU) performs approximate nearest-neighbor retrieval over both parameter shards and world-model entity embeddings in a unified index
- The BENDER FusionModule constructs the fused vector from retrieved world-model state, which is then injected via the Tokenformer mechanism

- The BENDER provenance chain provides an auditable trace of the coprocessor's influence on each inference step

This integration produces a unified pipeline: world-model knowledge stored persistently on SSD, retrieved efficiently by the CTU, structured by the BENDER cognitive runtime, and injected natively into the model's latent stream via Tokenformer. This is the target architecture for full system deployment.

8.3 Current Status and Research Path

The current implementation operates at Level 1 (structured control mode), as demonstrated by the Spider 2 result. Level 2 integration via Tokenformer adapter activation is in active development on the ScalarLM branch. Level 3 native residual injection and the full Dijkstra/BENDER pipeline constitute the primary research objectives.

Key open research questions include:

- What is the optimal injection point(s) for the fused vector in Qwen3.5's MoE architecture?
 - How should the FusionModule be trained or calibrated to align the vector space of graph-side state with the model's residual stream?
 - Does explicit provenance in the Control Packet correlate with measurable improvements in model consistency and factual accuracy, and at what granularity?
 - Can BENDER's hard constraint enforcement be shown to reduce a measurable class of generation errors in safety-relevant domains?
-

9. Conclusion

BENDER proposes a principled separation between structured runtime reasoning and probabilistic language generation. By maintaining an explicit typed world model and a deterministic cognitive runtime, it enables properties — provenance, hard constraint enforcement, consistency checking, and auditability — that are not reliably achievable through prompt-side methods or fine-tuning alone.

The architecture is grounded in a demonstrated benchmark result at Level 1 and a concrete integration path toward native latent influence via ScalarLM and the Dijkstra system. Domain instantiations in text-to-SQL, biomedical interpretation, and mass spectrometry illustrate the range of settings where structured runtime cognition outperforms approximate probabilistic retrieval.

The primary research objective is native request-scoped integration: guiding inference with structured world-model state in a way that is efficient, updateable, portable across model backends, and auditable by construction. We believe this constitutes both a meaningful

engineering contribution and a set of open empirical research questions with direct relevance to the deployment of safe and reliable AI systems.

References

- Alayrac, J.-B., et al. (2022). Flamingo: a Visual Language Model for Few-Shot Learning. *NeurIPS 2022*.
- Borgeaud, S., et al. (2022). Improving Language Models by Retrieving from Trillions of Tokens. *ICML 2022*.
- Edge, D., et al. (2024). From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *arXiv:2404.16130*.
- Guo, T., et al. (2024). Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows. *arXiv:2411.07763*.
- Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS 2020*.
- Li, X. L., and Liang, P. (2021). Prefix-Tuning: Optimizing Continuous Prompts for Generation. *ACL 2021*.
- Schick, T., et al. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. *NeurIPS 2023*.
- Sun, Z., et al. (2019). Rotate: Knowledge Graph Embedding by Relational Rotation in Complex Space. *ICLR 2019*.
- Wang, B., et al. (2021). KEPLER: A Unified Model for Knowledge Embedding and Pre-trained Language Representation. *TACL 2021*.